3.2 Bazy Wektorowe: Implementacja i przeszukiwanie semantyczne w MongoDB Atlas Vector Search

Moduł 3: Budowa Aplikacji i Agenty AI

Krzysztof Rytlewski · 2026

Czym są wektory i embeddingi?

Modele językowe przetwarzają tekst jako liczby. **Embedding** to proces zamiany dowolnego fragmentu tekstu na **wektor liczbowy** – tablicę liczb zmiennoprzecinkowych o stałej długości (np. 1536 wartości dla modelu `text-embedding-3-small` OpenAI).

Kluczowa właściwość: podobne znaczeniowo fragmenty mają podobne wektory. Zdania *„Jak wdrożyć Dockera?"* i *„Deployment przy użyciu kontenerów"* będą blisko siebie w przestrzeni wektorowej, nawet jeśli nie mają wspólnych słów.

Wektory możemy wyobrażać sobie jako punkty w przestrzeni wielowymiarowej – miara podobieństwa między nimi to kąt lub odległość między tymi punktami.

Miary podobieństwa między wektorami

Trzy główne miary podobieństwa stosowane w bazach wektorowych:

- **Cosine similarity** – mierzy kąt między wektorami (wartość od -1 do 1; 1 = identyczne). Najczęściej stosowana przy embeddingach tekstowych, ponieważ uniezależnia wynik od długości tekstu.

- **Dot product** – iloczyn skalarny; szybszy obliczeniowo, często stosowany w praktyce przy znormalizowanych wektorach.

- **Euclidean distance** – odległość euklidesowa; rzadziej przy embeddingach tekstowych.

MongoDB Atlas Vector Search domyślnie obsługuje `cosine` i `dotProduct`.

Po co dedykowana baza wektorowa?

Zwykłe bazy danych nie są przystosowane do wyszukiwania „najbliższych sąsiadów" w przestrzeni wielowymiarowej. Pełne przeszukanie milionów wektorów (brute-force) byłoby zbyt wolne.

Bazy wektorowe stosują **indeksy ANN** (Approximate Nearest Neighbor), które drastycznie skracają czas wyszukiwania kosztem minimalnej utraty precyzji:

Algorytm	Opis	Stosowany w
HNSW	Hierarchical Navigable Small World – graf warstwowy	MongoDB, Qdrant, Weaviate
IVF	Inverted File Index – klastry + centroids	Faiss, wyszukiwanie na dużą skalę
Flat	Brute-force (brak aproksymacji)	Małe kolekcje

MongoDB Atlas Vector Search – konfiguracja indeksu

MongoDB Atlas od wersji 6.0 wbudował obsługę wektorów bezpośrednio w silnik bazy danych. Konfiguracja jako indeks JSON:

```
{
  "fields": [
    {
      "type": "vector",
      "path": "embedding",
      "numDimensions": 1536,
      "similarity": "cosine"
    },
    {
      "type": "filter",
      "path": "category"
    }
  ]
```

Pole `numDimensions` musi odpowiadać wymiarowości używanego modelu. Pole `filter` pozwala zawężać wyniki bez wpływu na indeks wektorowy.

Tworzenie i zapisywanie embeddingów w Node.js

Typowy przepływ przy budowie bazy wiedzy:

```
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

async function embedAndStore(texts) {
  for (const text of texts) {
    const response = await openai.embeddings.create({
      model: 'text-embedding-3-small',
      input: text,
    });
    const embedding = response.data[0].embedding; // tablica 1536 liczb

    await collection.insertOne({
      content: text,
      embedding,
      createdAt: new Date(),
    });
```

Optymalna długość fragmentu do indeksowania: 300–500 tokenów z 10–20% nakładaniem (overlap).

Przeszukiwanie semantyczne – operator `$vectorSearch`

Wyszukiwanie przez MongoDB Atlas Vector Search odbywa się przez agregację:

```
async function semanticSearch(queryText, limit = 5) {
  const response = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: queryText,
  });
  const queryEmbedding = response.data[0].embedding;

  return await collection.aggregate([
    {
      $vectorSearch: {
        index: 'vector_index',
        path: 'embedding',
        queryVector: queryEmbedding,
        numCandidates: 100,  // co najmniej 10x wartości limit
        limit,
      },
```

Parametr `numCandidates` – trade-off jakość vs. prędkość

`numCandidates` to liczba kandydatów sprawdzana przez algorytm ANN przed zwróceniem finalnych `limit` wyników. Jest to kluczowy parametr wpływający na jakość wyników:

- **Wyższa wartość** → lepsza jakość wyników, wolniejsze zapytanie.

- **Niższa wartość** → szybsze zapytanie, niższa jakość.

- **Zalecenie:** `numCandidates` powinno być **co najmniej 10× wartości `limit`**.

Na przykład: `limit: 5` → `numCandidates: 100`. Dla `limit: 20` → `numCandidates: 200` lub więcej.

Filtrowanie i wyszukiwanie hybrydowe

Pre-filter – zawęża wyszukiwanie wektorowe do konkretnej kategorii dokumentów:

```
{
  $vectorSearch: {
    index: 'vector_index',
    path: 'embedding',
    queryVector: queryEmbedding,
    numCandidates: 100,
    limit: 5,
    filter: { category: 'devops' },
  }
}
```

Wyszukiwanie hybrydowe łączy wynik wektorowy z wynikiem pełnotekstowym (Atlas Search `$search`). Oba wyniki są re-rankowane przez *reciprocal rank fusion* (RRF). Poprawia trafność gdy kluczowe są konkretne słowa kluczowe: numery wersji, nazwy własne, identyfikatory.

Integracja z aplikacją Angular + Node.js

W architekturze MEAN warstwa wektorowa działa po stronie backendu. Angular komunikuje się wyłącznie przez REST API:

```
[Angular UI] → HTTP POST /api/search { query: "..." }
     → [Node.js / Express]
         → OpenAI Embeddings API
         → MongoDB Atlas $vectorSearch
         → zwróć top-N fragmentów do Angular
```

```
// Angular service
search(query: string): Observable<SearchResult[]> {
  return this.http.post<SearchResult[]>('/api/search', { query });
}
```

Angular nigdy nie odpytuje MongoDB bezpośrednio – komunikacja zawsze przez Express backend.